Tarea 2
Objetos, datos y animación

Alberto | Trabajo individual | Godot 4.x | 16 de octubre de 2026

1. Diseño orientado a objetos con criterio

La misma simulación se puede organizar alrededor de objetos que poseen estado y métodos, o alrededor de colecciones de datos que un algoritmo recorre. El propósito de este trabajo es comparar esas decisiones con una carga equivalente, sin atribuir cualquier diferencia de tiempo a una superioridad universal de una arquitectura.

Composición significa construir una capacidad reuniendo otras piezas. Un personaje puede contener un recurso visual, un controlador, salud y un inventario. Se pueden reemplazar esas piezas sin convertir cada combinación de características en una nueva subclase. La composición requiere definir quién posee el estado y cómo se comunican sus partes; tampoco es gratuita.

Herencia establece una relación de especialización. En Godot, derivar de Node permite participar en el árbol y reutilizar su ciclo de vida. Una jerarquía corta con contratos claros puede ser útil. Una jerarquía profunda puede acoplar reglas de combate, movimiento y representación: cambiar un método de una clase base afecta a muchas clases y obliga a comprender comportamiento repartido entre varios niveles.

Un objeto con demasiadas responsabilidades acumula razones independientes para cambiar. Si el jugador gestiona guardado, cámara, audio, animación y reglas del nivel, probar o reutilizar sólo su movimiento se vuelve difícil. Dividirlo permite aislar pruebas, pero una fragmentación excesiva añade comunicación e indirección. El objetivo es elegir límites útiles según el juego, no prohibir la orientación a objetos.

La comparación implementada

Se conservan 1 000 entidades, una semilla, la regla de movimiento y una ruta visual compartida. La variante OO usa un Node por entidad, con posición, velocidad y advance(). La segunda guarda posiciones y velocidades en dos PackedVector2Array y actualiza sus elementos desde un bucle central. Ambos diseños conviven con objetos del motor.

2. Datos, localidad y ECS

Indirección es acceder a un recurso a través de una referencia, un identificador o una llamada intermedia. Facilita sustituir implementaciones y compartir recursos. Seguir referencias entre muchos objetos puede añadir accesos y llamadas; su coste real depende del lenguaje, del motor y del patrón de acceso. No se mide contando únicamente líneas de código.

Localidad de referencia describe reutilizar datos próximos en el tiempo (localidad temporal) o recorrer elementos próximos en la memoria (espacial). Separar posiciones y velocidades de otros atributos concentra los datos que necesita el movimiento. La documentación describe los packed arrays como almacenamiento compacto [1]. Esto no demuestra por sí solo el número de fallos de caché del experimento.

Concepto	Definición y aplicación
Entidad	Identidad de un elemento simulado. En OO es un nodo; en DATOS es un índice entre 0 y 999.
Componente	Estado de una capacidad. Posición y velocidad son ejemplos conceptuales; no es obligatorio representarlos como nodos.
Sistema	Algoritmo que procesa entidades con los datos necesarios. El bucle de movimiento es un ejemplo.

Un ECS separa identidad, componentes y sistemas y suele ofrecer mecanismos para seleccionar conjuntos de componentes y administrar sus ciclos de vida. La versión DATOS de este proyecto no dispone de registro de entidades, composición dinámica ni consultas de componentes. Es una organización orientada a datos de forma fija, no un framework ECS.

PhysicsServer3D permite gestionar objetos físicos mediante una API de bajo nivel, pero usarlo no crea un ECS ni demuestra una disposición contigua de cuerpos físicos. Este ensayo no lo necesita: no hay colisiones ni física en ninguna de las dos variantes. No se utiliza GDExtension ni un paquete ECS externo.

Equivalencia funcional antes de cronometrar

Cada actualización calcula p = p + v × delta y aplica wrapf a X en [0,1120) y a Y en [0,480). El cruce de un borde reaparece por el opuesto. La prueba scripts/verify.gd compara las 1 000 posiciones después de cada uno de 2 100 pasos de 1/60 s. La tolerancia es 0.001 píxeles; el máximo observado fue 0.0. Este control verifica también cruces de borde y que las posiciones sean finitas.

En las corridas reales se usa delta del motor. La duración real y la regla coinciden, pero los fotogramas y los estados finales pueden diferir por planificación temporal. Por eso la equivalencia se valida además con los mismos pasos fijos, y no mediante una comparación de capturas finales.

3. Método de medición

La suite ejecuta tres pares con orden alternado: OO-DATOS, DATOS-OO, OO-DATOS. Así cada variante tiene tres corridas y aparece en posiciones tempranas y tardías. Se crean de nuevo las entidades con semilla 20261016 al comenzar cada corrida. El coste de inicialización queda fuera del cronometraje.

Cada corrida deja pasar 5 s de calentamiento y después guarda muestras por fotograma durante 30 s, usando reloj monotónico Time.get_ticks_usec() [2]. Los CSV se acumulan en memoria y se escriben al finalizar; la E/S de cierre no pertenece al bloque instrumentado. Las mediciones no se hacen en modo headless ni con el grabador de video.

Condición	Valor registrado
Sesión analizada	2026-10-16T22-31-26
CPU	AMD Ryzen 7 5700X 8-Core Processor
GPU	NVIDIA GeForce RTX 4060
RAM física	15.93 GiB (dato de OS.get_memory_info)
Motor / sistema	4.7.2-stable (official) / Windows
Renderizador / driver	gl_compatibility / opengl3
Resolución / VSync	(1280.0, 720.0) / desactivado (0)
Carga	1 000 círculos de radio 2.4 px; mismo CanvasItem y materiales de color
Ejecución	Binario oficial con debug_build=true; sin profiler conectado durante las corridas

Ambas variantes copian 1 000 posiciones a un buffer visual y llaman al mismo _draw(), que emite 1 000 círculos. El número de callbacks de dibujo queda registrado. El controlador actualiza su rótulo cuatro veces por segundo. No se compara una variante sin dibujar contra otra que sí dibuja. El experimento deliberadamente no compara 1 000 CanvasItems frente a batching: eso mezclaría la organización de datos con otra estrategia visual.

Las seis corridas son secuenciales y comparten proceso; no equivalen a seis equipos ni a seis arranques independientes. La sesión de desarrollo preliminar se conserva fuera del conjunto analizado. La suite final se ejecuta después de preparar los recursos y videos, sin esas tareas pesadas simultáneas. No se controlan temperatura, frecuencia de CPU, servicios del sistema o limitadores externos del controlador gráfico.

4. Datos y unidades

El CSV original conserva frame, elapsed_s, delta_ms, frame_wall_ms, movement_block_ms, snapshot_ms y draw_count. Los tiempos de duración usan segundos; los tiempos de rendimiento usan milisegundos. Cada JSON adjunto conserva hardware, motor, configuración y cantidad de muestras.

El bloque instrumentado también usa reloj de pared: una interrupción o desplanificación del proceso puede aumentar su duración. No mide exclusivamente ciclos de CPU ejecutando GDScript.

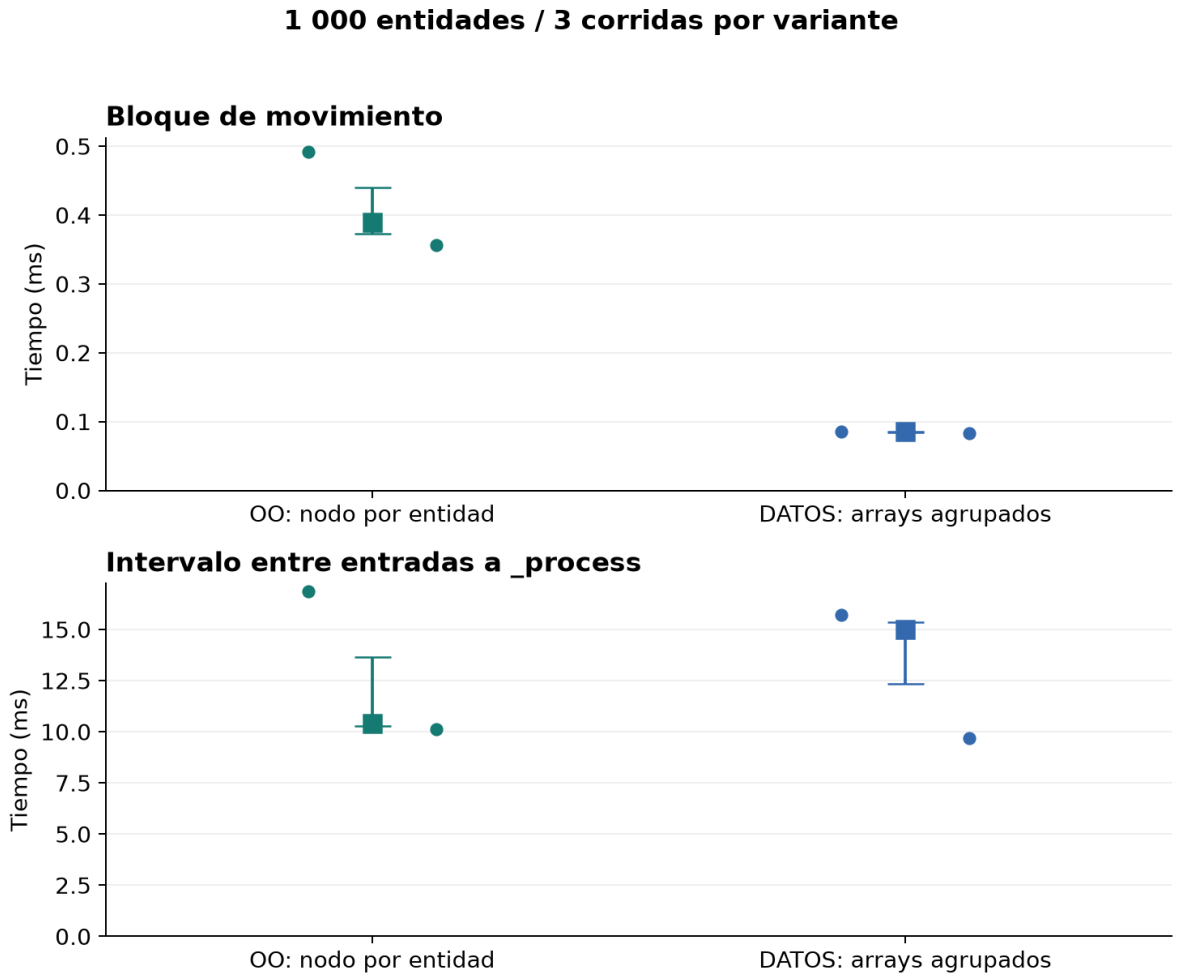
Métrica	Qué incluye y qué excluye
movement_block_ms	Reloj antes y después de advance(delta). Incluye bucle, acceso al estado y llamadas OO. Excluye copia visual, dibujo y GPU.
snapshot_ms	Copia explícita de 1 000 posiciones al buffer visual. Se registra separadamente.
frame_wall_ms	Intervalo de reloj entre entradas consecutivas a _process. Incluye efectos de planificación y esperas; no es tiempo exclusivo de CPU ni de GPU.
delta_ms	Delta que Godot suministra al movimiento. No sustituye al reloj de pared.
Profiler CPU	No capturado en esta suite. El profiler permite examinar funciones y tiempo de CPU con otra instrumentación [3]. No se renombra ninguna columna como CPU.

Mediana e IQR dentro de cada corrida

Corrida	Variante	Muestras	Movimiento ms	IQR ms	Fotograma ms	IQR ms
1	OO	1745	0.492	0.059	16.883	2.209
2	DATOS	1877	0.086	0.005	15.711	2.165
3	DATOS	1966	0.086	0.002	14.997	2.128
4	OO	2723	0.389	0.106	10.416	1.301
5	OO	2887	0.356	0.068	10.142	0.738
6	DATOS	3031	0.083	0.002	9.706	0.511

IQR = percentil 75 - percentil 25, con interpolación lineal. La tabla no mezcla todos los fotogramas como si fueran experimentos independientes: resume cada corrida primero. summary.csv agrega además Q1, Q3 y P95 por métrica. aggregate.csv resume después las tres medianas de cada variante. Las muestras consecutivas están correlacionadas y sólo hay tres repeticiones por variante; no se calculan significancias estadísticas.

5. Comparación gráfica



Puntos: medianas por corrida. Cuadrado: mediana de las 3 medianas. Barras: Q1-Q3 entre corridas.

Las escalas verticales son independientes para hacer legibles dos magnitudes distintas. Los puntos representan tres medianas, no miles de experimentos. Las barras muestran Q1-Q3 de esas tres medianas; el IQR por fotograma de cada corrida aparece en la página anterior. Un intervalo estrecho no demuestra ausencia de sesgo.

6. Interpretación y límites

La mediana de las medianas del bloque fue **0.389 ms en OO** y **0.086 ms en DATOS**. Para el intervalo de fotograma fue **10.416 ms** y **14.997 ms**, respectivamente. Estos son resultados de esta implementación, máquina y carga; no describen cualquier juego de 1 000 entidades.

En esta sesión, DATOS necesitó menos tiempo de movimiento: el cociente OO/DATOS es 4.52. Ese cociente pertenece sólo al bloque medido. No es una aceleración de todo el fotograma ni prueba que otro ECS, C# o C++ obtenga la misma relación.

El fotograma de DATOS fue un 44.0% mayor según esas medianas, pese al menor bloque de movimiento. Esto impide afirmar una mejora global. Las medianas por corrida también muestran un descenso temporal del intervalo de fotograma: las últimas corridas son más rápidas en ambas variantes. Esa deriva puede confundir el resumen por arquitectura; alternar el orden reduce sesgos, pero seis corridas no los elimina. No se identificó la causa de la deriva.

El resultado combina disposición de estado y coste de ejecución. OO llama advance() a través de cada objeto; DATOS realiza operaciones en un bucle. No se ha aislado el efecto exclusivo de memoria frente al de despacho de métodos. Tampoco se han medido fallos de caché, instrucciones, ancho de banda o consumo energético. No se atribuye una diferencia de milisegundos a cache misses no observados.

El dibujo, la copia visual, el motor y las esperas pueden dominar el intervalo de fotograma. Si cambia poco mientras el bloque cambia, no hay contradicción: el bloque es sólo parte del trabajo. VSync está desactivado, pero el sistema operativo o el controlador gráfico pueden introducir esperas. Una frecuencia próxima al refresco no basta para identificar su causa.

Los vectores packed usan precisión de Vector2 del motor. Ambas variantes aplican las mismas operaciones y el test de estado encontró error cero. El patrón es simple: velocidad constante, sin búsqueda de vecinos, altas/bajas, colisiones, inteligencia artificial ni jerarquías visuales por entidad. Los resultados no permiten extrapolar el coste de un juego con componentes heterogéneos.

Cómo ampliar el experimento

Un siguiente ensayo podría repetir más bloques de orden contrabalanceado, registrar temperatura y frecuencia, y usar el profiler en una sesión separada para localizar funciones costosas. También convendría evaluar cambios de número de entidades sin modificar simultáneamente la carga visual. La prioridad sería explicar una causa con evidencia, no buscar un porcentaje predeterminado.

7. Rig, clips y transiciones

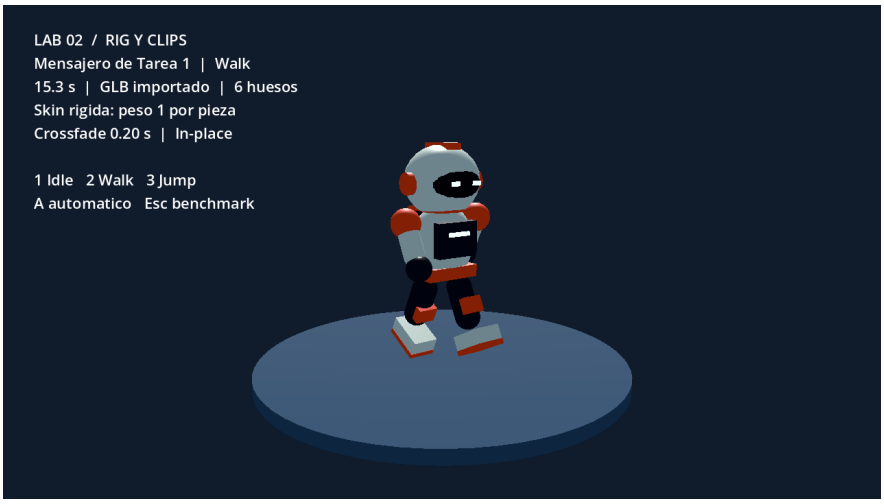
Se reutiliza el mensajero de SKYLINE de Tarea 1, del repositorio de Alberto, commit 3800517dbe27de3c0f72e1b68a289f06d8bea216 [4]. El recurso de entrada y su procedencia se preservan en assets/source. La herramienta de autoría es Godot 4.7.2 mediante tools/build_rig.gd; no se utilizó Mixamo.

El rig tiene seis huesos: Root, LeftArm, RightArm, LeftLeg, RightLeg y Scarf. Se hornean las transformaciones de las 30 piezas de la escena original en la pose de enlace. Cada vértice recibe peso 1 en el hueso de su pieza: es skinning rígido, apropiado para este personaje segmentado, sin deformación blanda de codos o rodillas. El recurso se exporta a GLB y se vuelve a importar en Godot; la escena de prueba carga ese GLB [5].

Idle (1.0 s) mueve sutilmente brazos y bufanda; Walk (1.0 s) alterna brazos y piernas ±0.48 rad; Jump (1.2 s) eleva brazos y flexiona piernas. Son clips in-place con rotaciones, sin pista de traslación de raíz. El controlador character_lab.gd selecciona el clip y mezcla durante 0.20 s; el recurso visual contiene mallas, esqueleto y AnimationPlayer. La demostración mantiene el personaje en su lugar, evitando doble desplazamiento.

Prueba renderizada de más de 60 s: Idle 20.11 s, Jump 20.00 s, Walk 20.00 s. Se registraron 6 transiciones; poses finitas: True; deriva máxima del nodo visual: 0.000000 m.

Tres verificaciones: (1) el GLB importado contiene seis huesos y los tres nombres de clips; (2) las poses de todos los huesos permanecen finitas durante la prueba; (3) la raíz no se traslada. Las capturas y el video permiten además inspeccionar que las piezas conservan su volumen. Se normalizaron las normales con la matriz inversa transpuesta al hornear escalas no uniformes; las tangentes anteriores se descartan porque no hay normal maps.



8. Reproducción y fuentes

Abrir project.godot y ejecutar F5 muestra el benchmark; el botón inicia las seis corridas. La escena scenes/character.tscn abre el laboratorio de animación. El README contiene comandos de importación, test de equivalencia, suite, análisis, informe y video. No se requieren extensiones de Godot ni recursos descargados al ejecutar el proyecto.

Los datos originales quedan en results; las tablas y las figuras PNG/SVG, en output/analysis; los videos separados y los controles de animación, en output/evidence. tools/analyze.py es la fuente editable de las gráficas y tools/build_report.py la del PDF. El número de muestras varía al repetir la suite: se esperan archivos utilizables y condiciones iguales, no resultados numéricos idénticos.

Procedencia y asistencia de IA

Trabajo preparado para entrega individual de Alberto. Se declara asistencia de Codex para el código GDScript/Python, la adaptación del rig, la ejecución automatizada, el análisis y el borrador del informe. El personaje original también declaraba asistencia de IA. No se ejecutó un generador de imágenes o modelos en esta tarea. La consigna y el enlace al repositorio son las entradas de esta sesión; el prompt histórico se conserva sin modificar en assets/source/prompt_tarea1.txt. Alberto debe revisar y comprender el material antes de presentarlo como su entrega.

Fuentes consultadas

[1] Godot. PackedVector2Array. Almacenamiento compacto y semántica de referencia.
https://docs.godotengine.org/en/stable/classes/class_packedvector2array.html

[2] Godot. Time. Relojes monotónicos get_ticks_usec.
https://docs.godotengine.org/en/stable/classes/class_time.html

[3] Godot. The Profiler. Instrumentación y lectura de tiempos.
https://docs.godotengine.org/en/stable/tutorials/scripting/debug/the_profiler.html

[4] Alberto415. Tarea 1: fuente del mensajero y licencia CC0 declarada.
https://github.com/Alberto415/Tarea-1-Escena-base-y-modelo-generado-con-IA

[5] Godot. Skeleton3D. Pose de reposo, huesos y skin.
https://docs.godotengine.org/en/stable/classes/class_skeleton3d.html

[6] Godot. Creating movies. Captura a paso fijo; no usarla para evaluar rendimiento real.
https://docs.godotengine.org/en/stable/tutorials/animation/creating_movies.html

Laboratorio data orientation de la UAM: https://ubicua.cua.uam.mx/mod/resource/view.php?id=150718. Se intentó abrir, pero no fue accesible desde esta sesión. No se atribuyen a ese material instrucciones o resultados que no se pudieron consultar. Las fuentes stable pueden cambiar; la versión utilizada en la medición queda registrada en cada JSON.